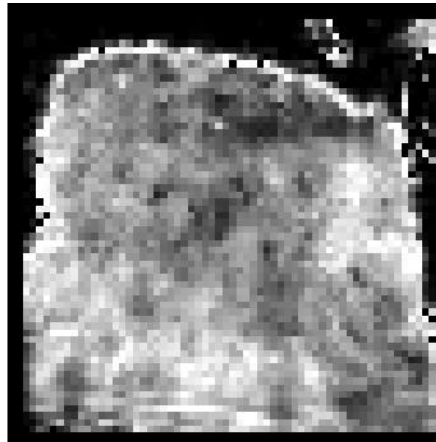


Segmentazione (di basso livello)



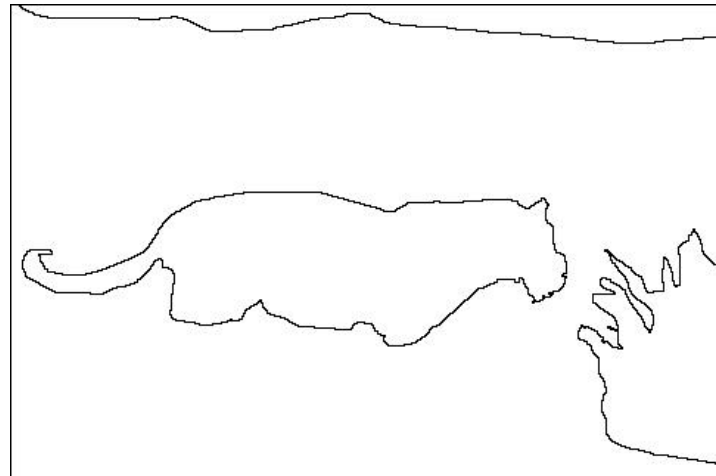
Segmentazione

La segmentazione può avere obiettivi diversi a seconda della specifica applicazione.



separazione di uno o più
oggetti di interesse dallo
sfondo

Segmentazione: superpixel



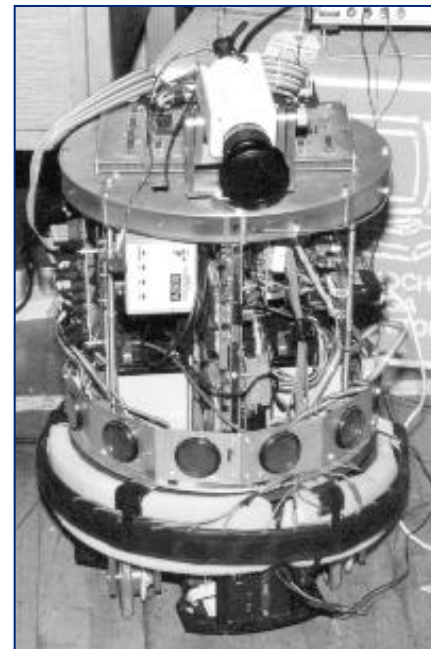
partizionamento
dell'immagine in regioni
con proprietà comuni

Segmentazione oggetto/sfondo

Può essere un compito a volte semplice ma talvolta anche molto **complesso**:



Caso Semplice



Caso complesso !

Binarizzazione con soglia/e globale/i

- Nel caso di una singola soglia globale T (es. un solo oggetto), il metodo consiste nell'impostare a 0 i pixel la cui intensità è inferiore a T e a 1 i rimanenti.

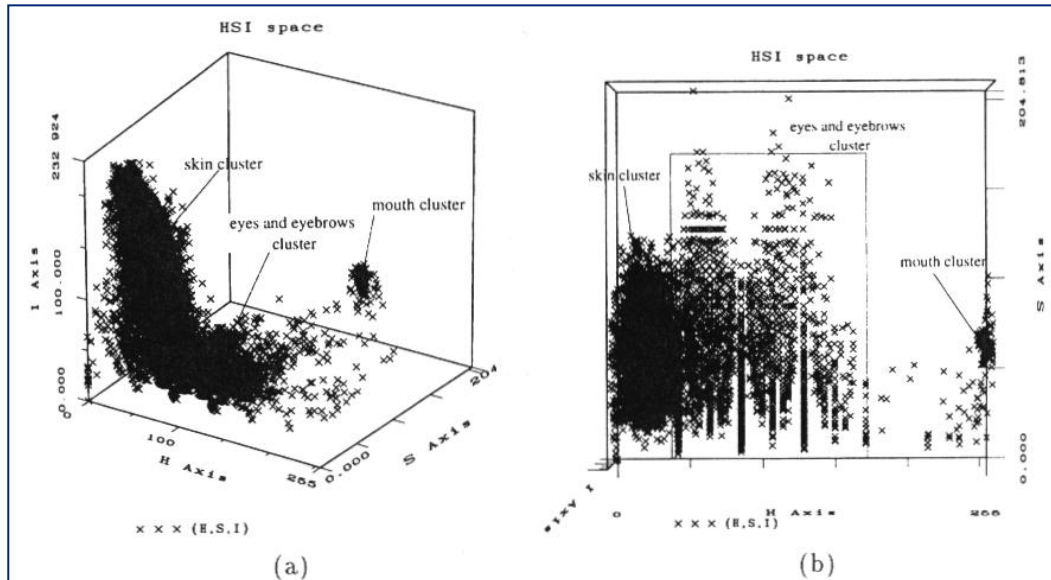
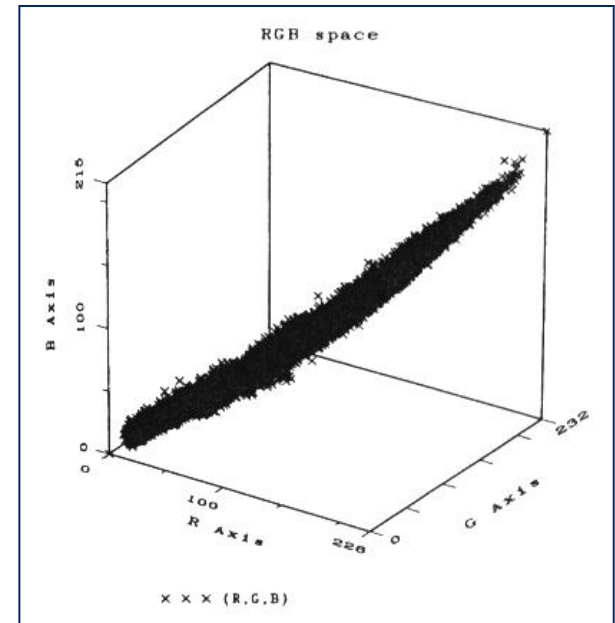
$$\text{New}I[x, y] = \begin{cases} 0 & I[x, y] < T \\ 1 & I[x, y] \geq T \end{cases}$$

- Sebbene molto semplice il metodo presenta i **seguenti problemi**:
 - scelta automatica della soglia/e ottimale/i
 - non corretta gestione di sfondi a intensità variabile (es. gradienti luce).
 - incapacità di segmentare oggetti non uniformi
- Altri metodi, ad es. Otsu, scelgono dinamicamente la soglia, in modo da massimizzare la distanza tra le due classi.

Segmentazione colore: esempio

Localizzazione del **volto** e delle sue componenti interne:

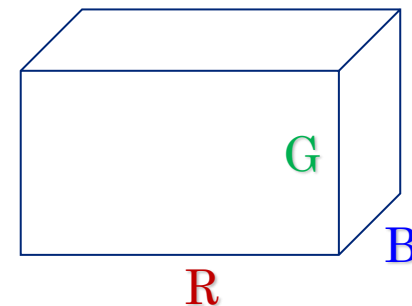
- nello spazio RGB le diverse regioni del volto non sono spazialmente raggruppate, mentre in HSI si notano **cluster** ben evidenti per **bocca**, **occhi**, **pelle**, ... e quindi è relativamente semplice isolare tali caratteristiche.



Segmentazione colore (1)

□ L'utilizzo di **tecniche empiriche** per estrarre le regioni dello spazio colore che contengono gli oggetti che si vogliono segmentare non è raccomandabile (tranne nei casi molto semplici) in quanto:

- risulta **difficilmente** **generalizzabile**, costringendo a noiose e critiche tarature manuali se le condizioni (luce, tipo di immagine, ecc...) cambiano.
- l'impiego di **parallelepipedi** per racchiudere l'insieme di pixel di interesse (che porta a ottenere range indipendenti di ammissibilità per ciascuna componente dello spazio colore, es. $50 < R < 60$, $100 < G < 120$, $90 < B < 140$), non porta a risultati molto accurati.



Segmentazione colore (2)

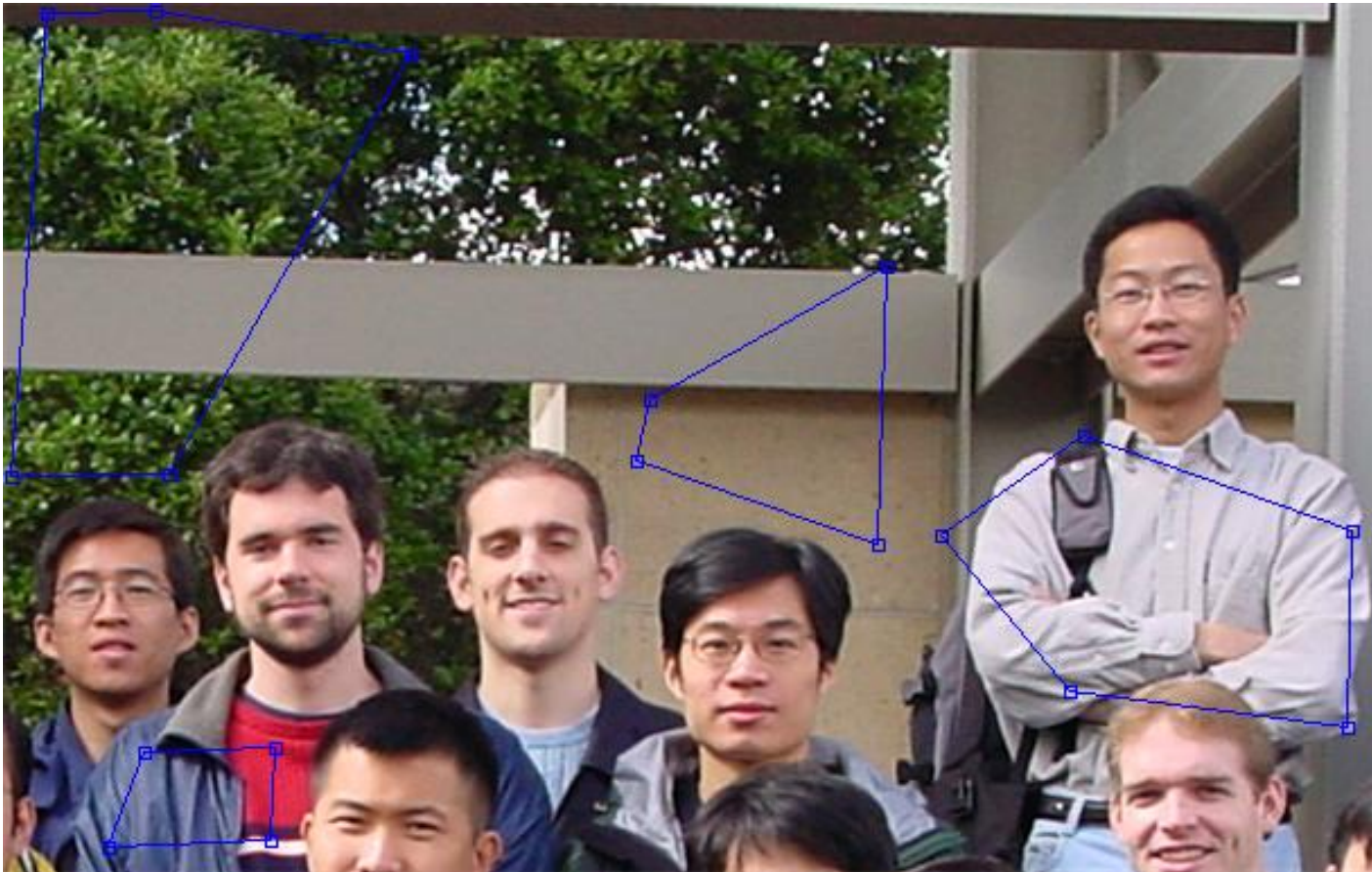
- ❑ Molto meglio **etichettare** manualmente (ad esempio racchiudendo con poligoni) le regioni di interesse su alcune immagini di training e lasciare a un **classificatore** il compito di individuare la sottoregione (anche non connessa) dello spazio colore. Terminato il training, il classificatore addestrato potrà essere utilizzato per segmentare nuove immagini:
 - il procedimento può essere agevolmente ripetuto anche se le condizioni cambiano (sufficiente etichettare qualche esempio anche delle nuove immagini).
 - i classificatori possono derivare regioni di forma anche molto complessa (attenzione però alla generalizzazione ...).

Esempio con classificatore



Pixel appartenenti al volto (foreground) definiti come pixel interni ai 3 poligoni marcati a mano

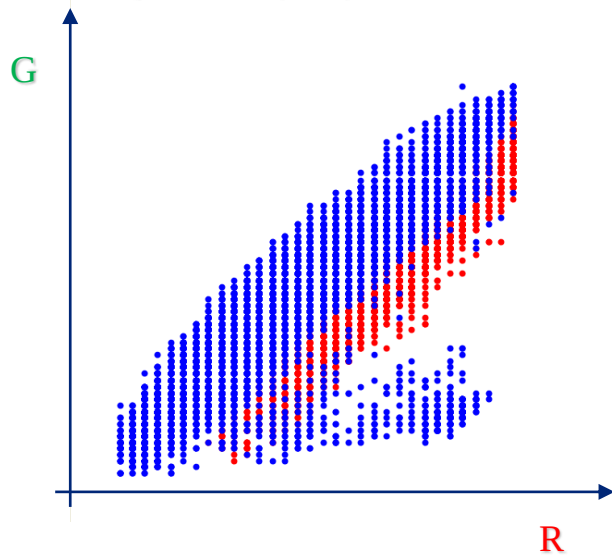
Esempio con classificatore



Pixel non appartenenti al volto (background) definiti
come pixel interni ai 4 poligoni marcati a mano

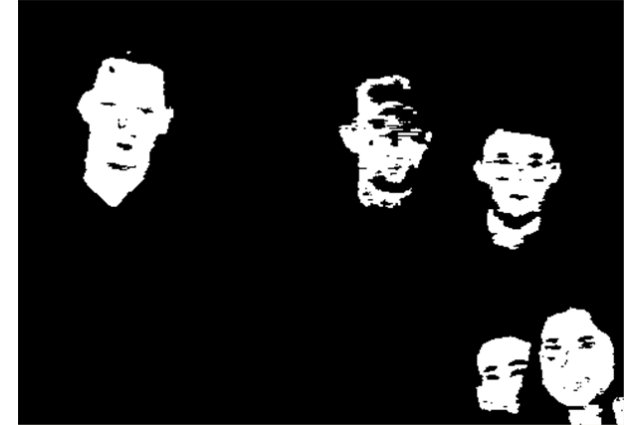
Esempio con classificatore

I pixel del training set in una vista 2D (RG)
dello spazio RGB: **foreground (rossi)**
background (blu)



Un classificatore bayesiano
parametrico con distribuzioni
multinomiali è addestrato a
separare le due classi in RGB
e applicato a una nuova
immagine

Risultato



Risultato sovrapposto all'immagine

Esempio verde pubblico



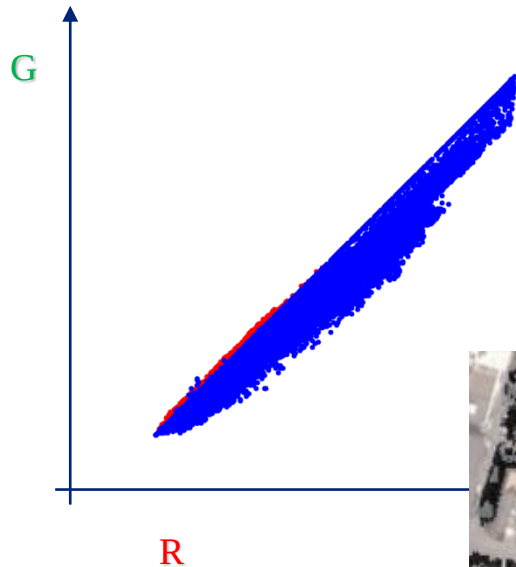
Verde Pubblico in un quartiere di Cesena

Esempio verde pubblico



Non “Verde Pubblico” sullo stesso quartiere

Esempio verde pubblico

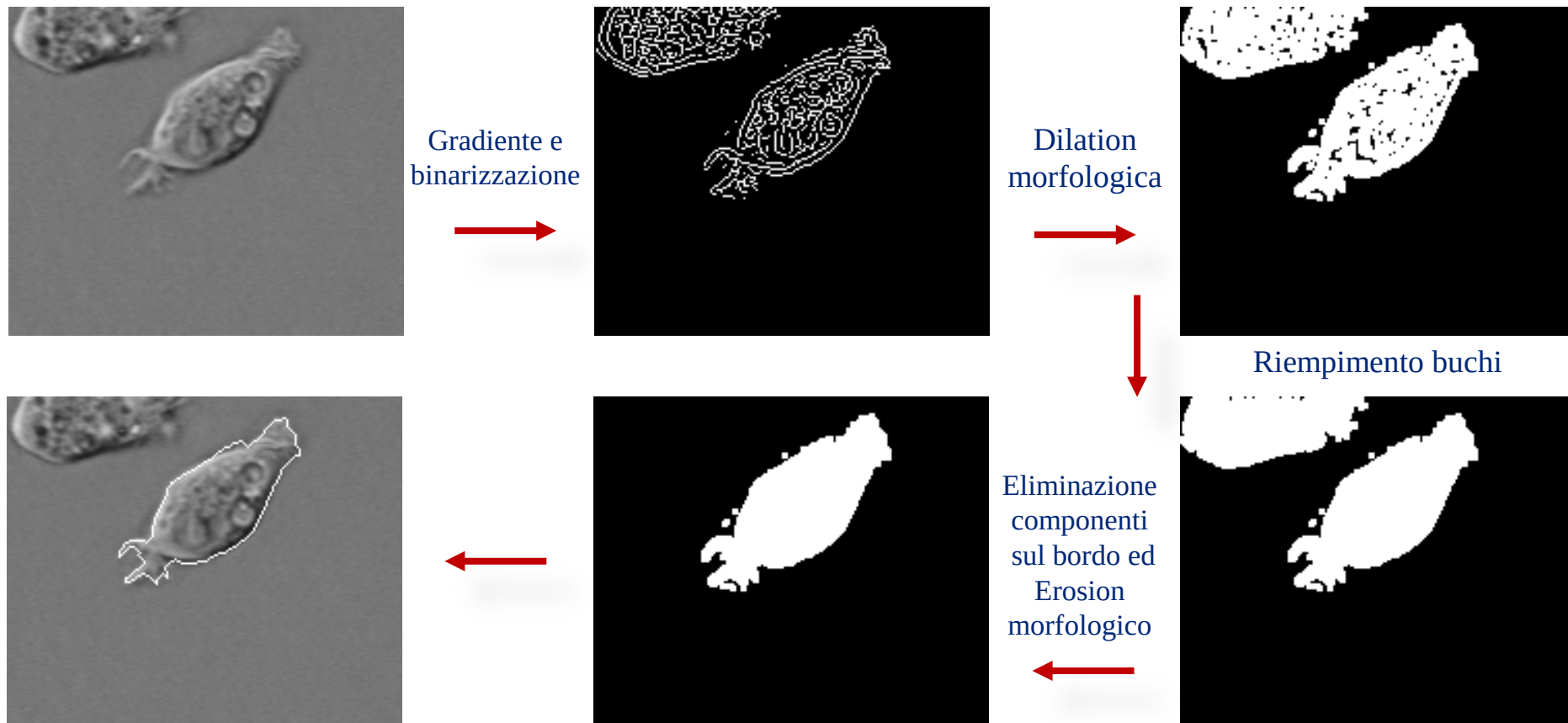


Un classificatore bayesiano parametrico con distribuzioni multinormali è addestrato a separare le due classi in RGB e applicato a una nuova immagine



Impiego di edge per la segmentazione

Nei casi in cui lo sfondo è sufficientemente uniforme ma gli oggetti non sono caratterizzati da intensità (grigi) o colori uniformi, l'impiego di edge può aiutare nella segmentazione.



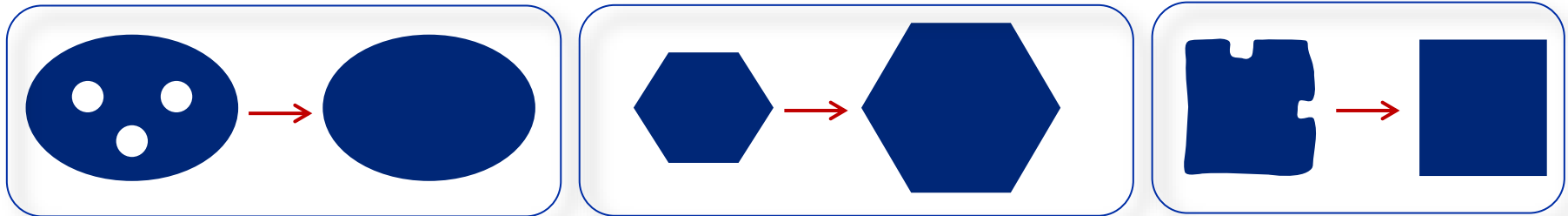
Regolarizzazione della segmentazione

- La maschera binaria ottenuta è talvolta irregolare
- Può essere regolarizzata utilizzando appositi operatori detti operatori morfologici

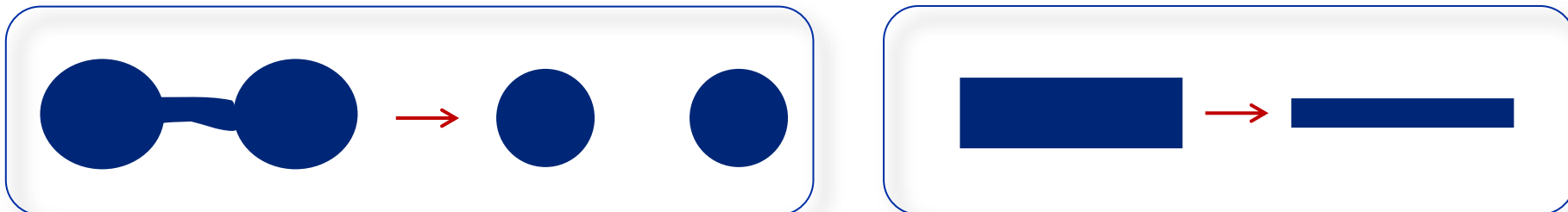


Morfologia matematica

- La morfologia matematica, derivata dalla teoria degli insiemi, offre strumenti che consentono di estrarre informazioni utili per rappresentare la forma degli oggetti e rimuovere particolari irrilevanti.
- Lavora solitamente su immagini binarie (Foreground/Background).
- Esistono due principali operatori morfologici:
 - **Dilatazione** (dilation): **espande** gli insiemi connessi di pixel di foreground (aumentare le dimensioni degli oggetti, riempire buchi o zone vuote).



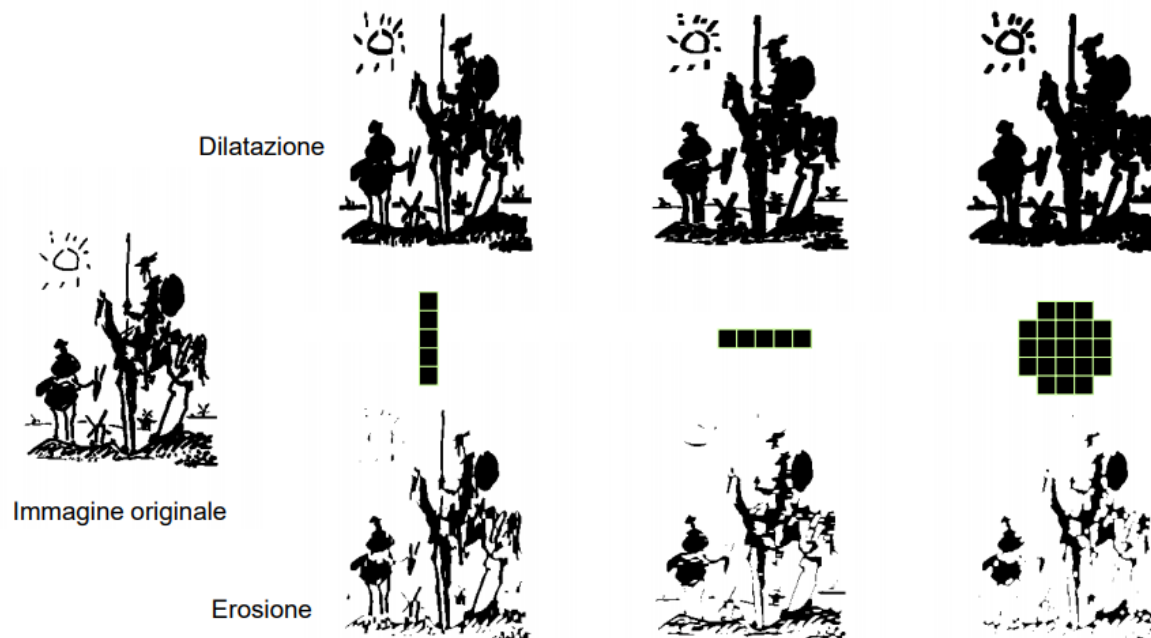
- **Erosione** (erosion): **contrae** gli insiemi connessi di pixel di foreground (assottigliare gli oggetti, separa oggetti debolmente connessi).



L'elemento strutturante



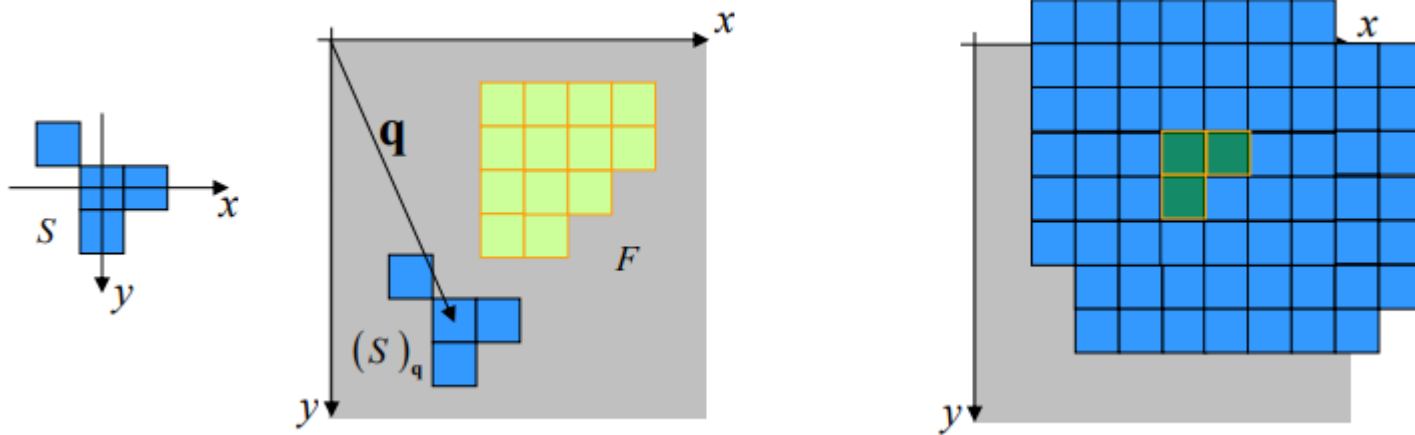
- Gli operatori morfologici modificano il contenuto dell'immagine binaria applicando un *elemento strutturante S* che può avere una qualsiasi forma rappresentabile tramite una maschera digitale.
- Solitamente si utilizzano elementi strutturanti *circolari* o *quadrati*.



Erosione

- Data un'immagine binaria F e un elemento strutturante S , l'immagine risultante avrà valore di foreground nei pixel tali che, traslando in essi S , l'intero elemento strutturante è contenuto in F :

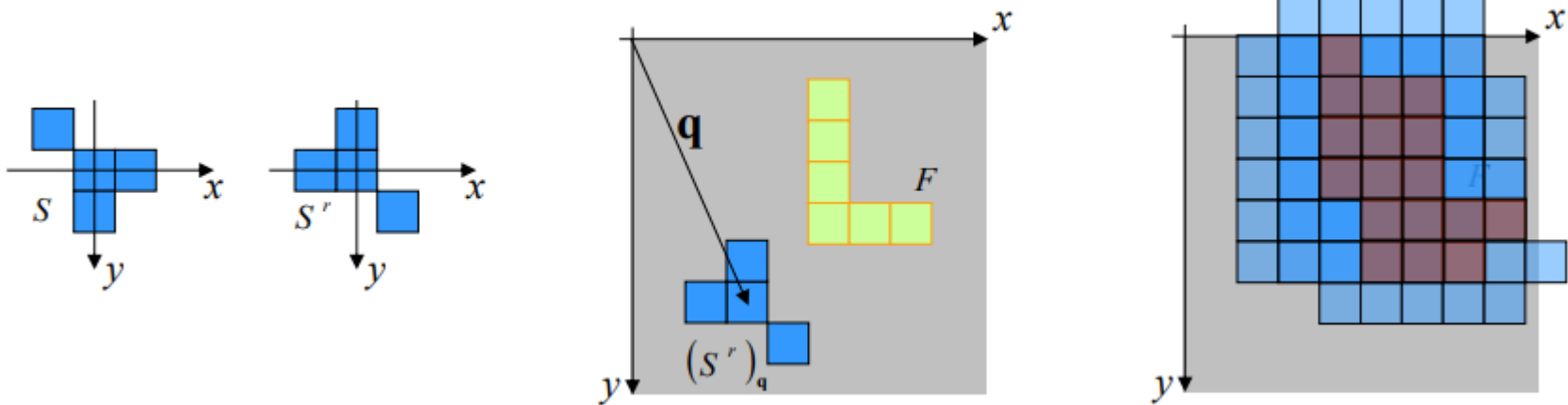
$$F \ominus S = \{q | (S)_q \subseteq F\}$$



Dilatazione

- Data un'immagine binaria F e un elemento strutturante S , l'immagine risultante avrà valore di foreground nei pixel tali che, traslando in essi S^r , almeno uno dei suoi elementi è contenuto in F :

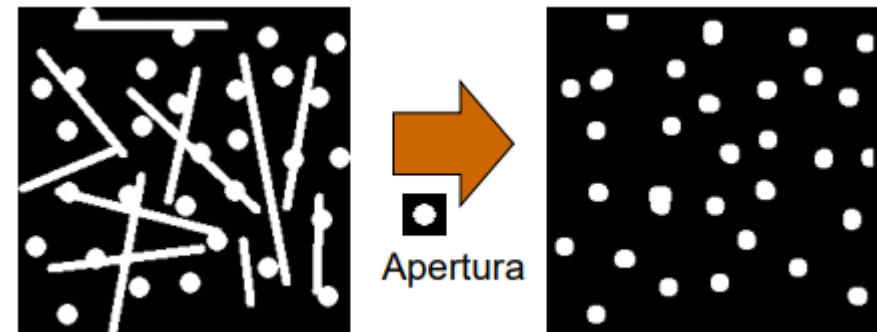
$$F \oplus S = \{q | (S^r)_q \cap F \neq \emptyset\}$$



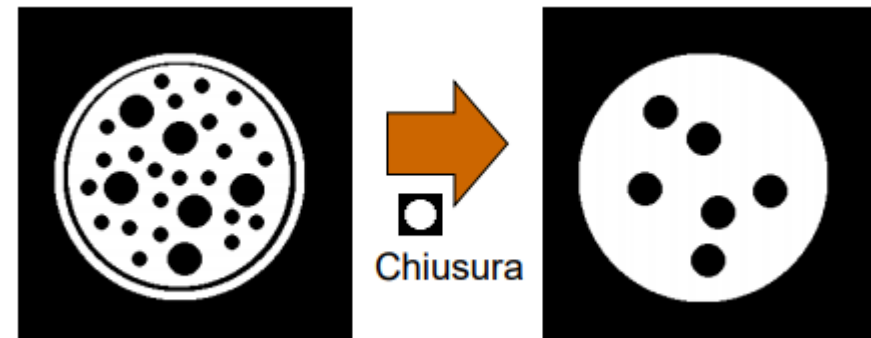
Opening e closing

Combinando erosione e dilatazione si possono ottenere altre operazioni:

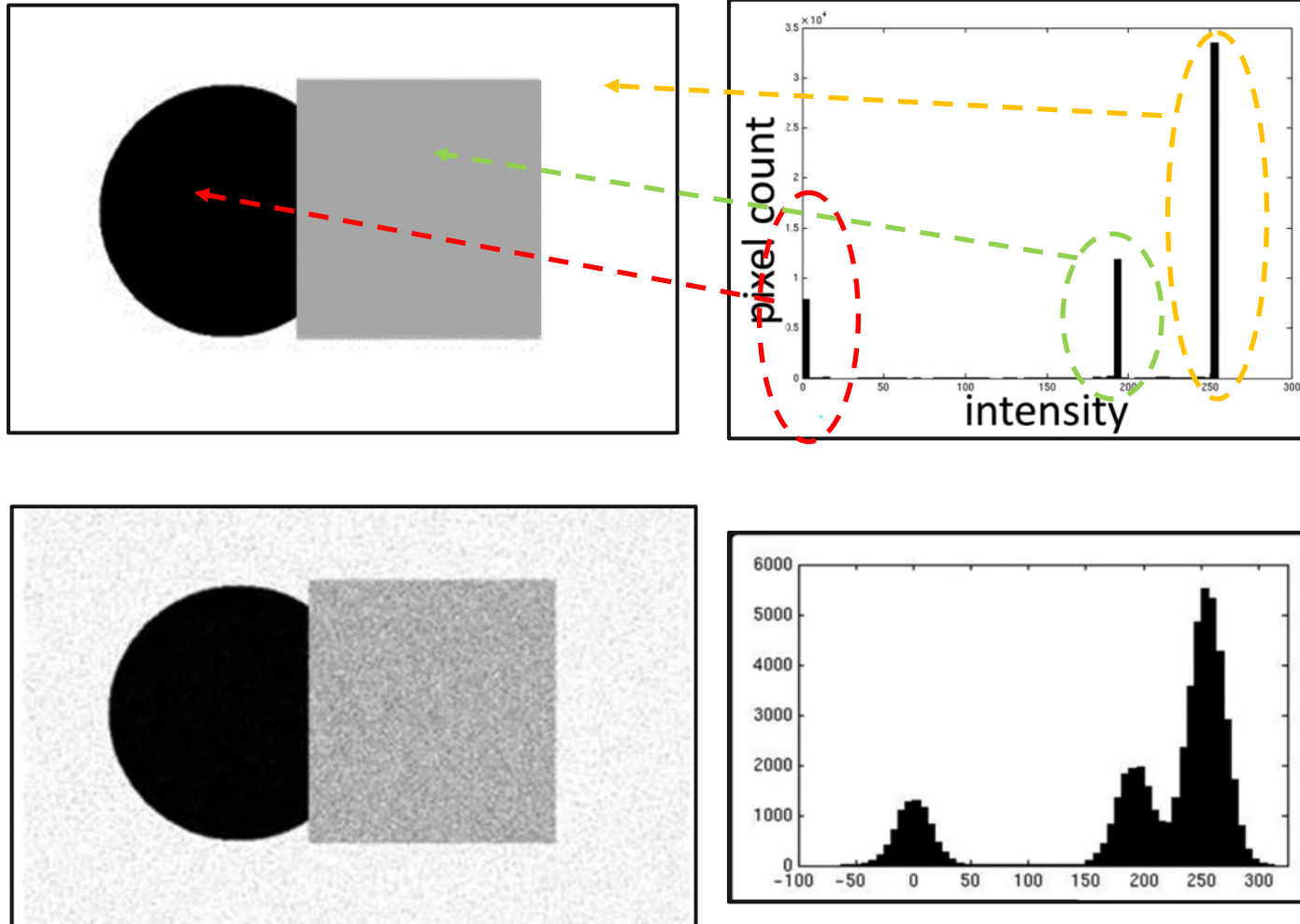
- **Apertura** (opening):
erosione + dilatazione
 - Separa oggetti debolmente connessi e rimuove regioni piccole



- **Chiusura** (closing):
dilatazione + erosione
 - Riempie buchi e piccole concavità e rafforza la connessione di regioni unite debolmente



Segmentazione in presenza di rumore



Segmentazione tramite clustering (1)

- Gli oggetti di interesse possono essere individuati applicando **algoritmi di clustering** (es. K-Means).
- La scelta dello spazio delle feature è fondamentale.
- Supponiamo di utilizzare solo il valore di **intensità**.



I migliori centroidi sono quelli che minimizzano la *Sum of Squared Distances*:

$$SSD = \sum_{cluster\ C_i} \sum_{p_j \in C_i} \|p_j - c_i\|$$

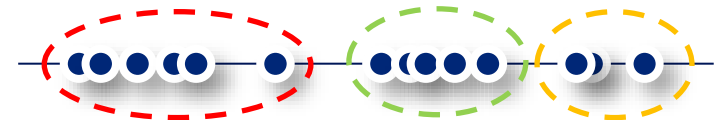


K-means clustering

Conoscendo i centroidi, quali punti saranno assegnati a ogni cluster?



Conoscendo l'attribuzione ai cluster come determinare i centroidi?



Algoritmo

1. Inizializza casualmente i centri dei cluster
2. Assegna i punti ai cluster:
 - Per ciascun punto p , trova il cluster più vicino c_i e assegna p al cluster c_i
3. Sulla base dell'assegnamento effettuato al passo 2 ricalcola i centroidi di ciascun cluster:
 - Calcola c_i come media dei punti assegnati al cluster
4. Se i cluster sono stati modificati, ripeti dal passo 2.

Segmentazione tramite clustering (1)

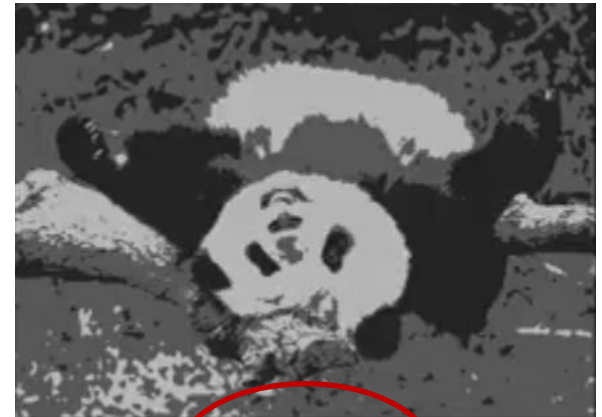
- Gli oggetti di interesse possono essere individuati applicando **algoritmi di clustering** (es. K-Means).
- La scelta dello spazio delle feature è fondamentale.
- Supponiamo di utilizzare solo il valore di **intensità**.



Feature space 1D

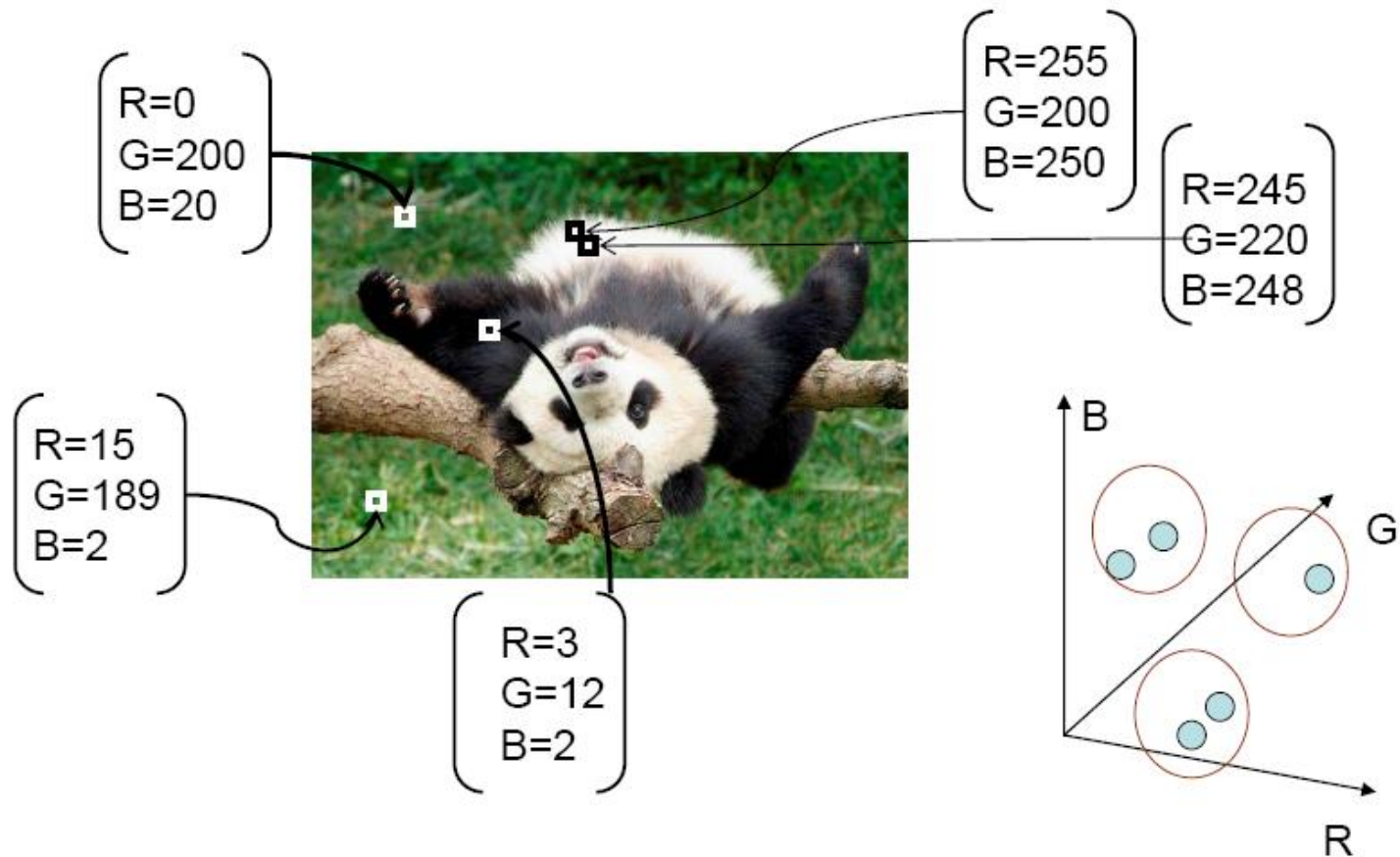


$K = 2$



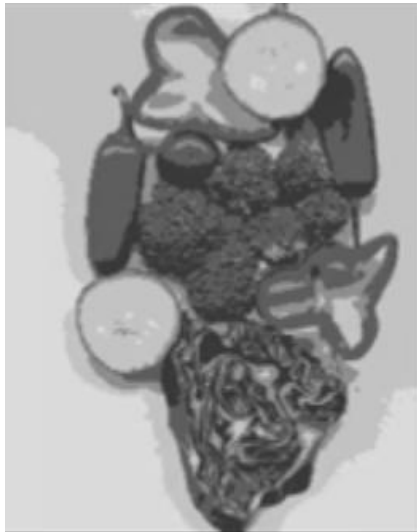
$K = 3$

Segmentazione tramite clustering (2)



Segmentazione tramite clustering (3)

- Un feature space più **rappresentativo** consente di ottenere risultati migliori



solo intensità

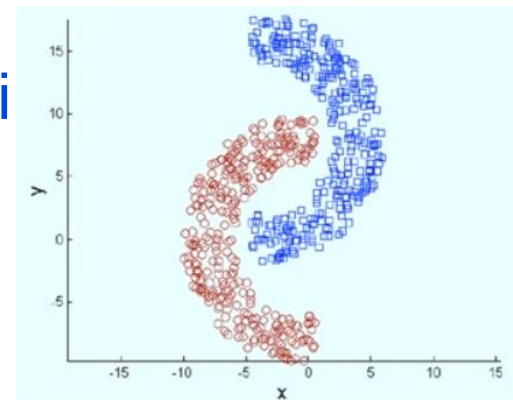


Colore (RGB)

- Problema: oggetti distanti possono essere assegnati allo stesso cluster...
- Soluzione: codificare nello spazio delle feature anche **informazioni spaziali** (ma anche tessitura, gradiente, ecc...)

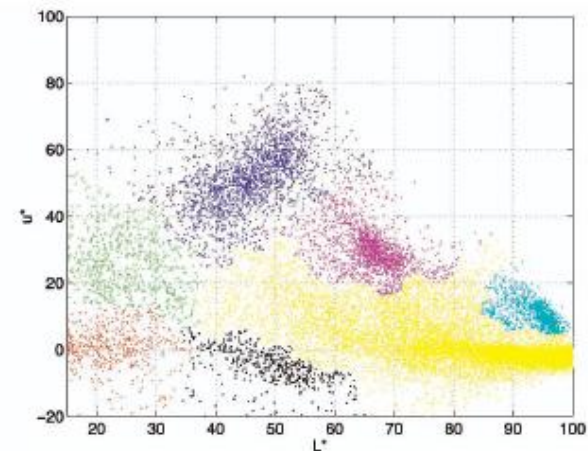
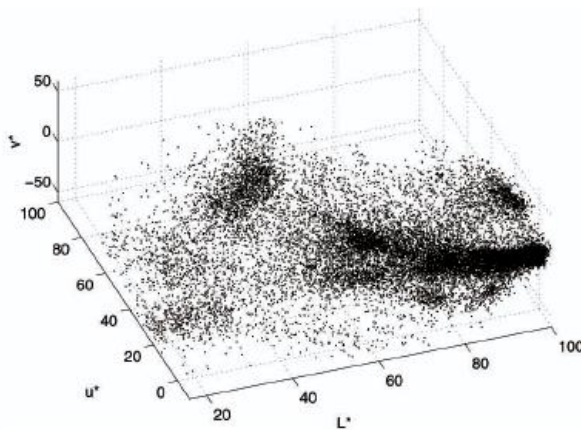
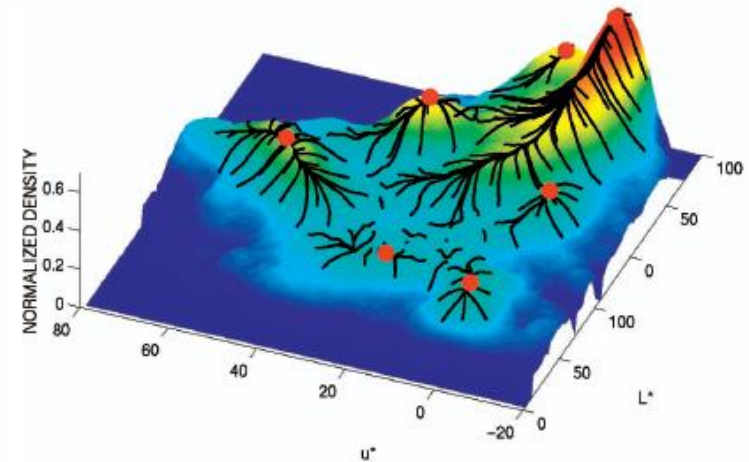
K-means: pro e contro

- Pro:
 - Algoritmo molto **semplice**;
 - Converge a un **minimo locale** della funzione errore;
- Contro:
 - Elevata **complessità computazionale**;
 - Necessità di definire il **valore di k**;
 - Sensibile alla **scelta iniziale** dei centroidi
 - Sensibile alla presenza di **outlier**;
 - Individua cluster di forma “**sferica**” (non ottimale per alcune distribuzioni di pattern)



Mean shift

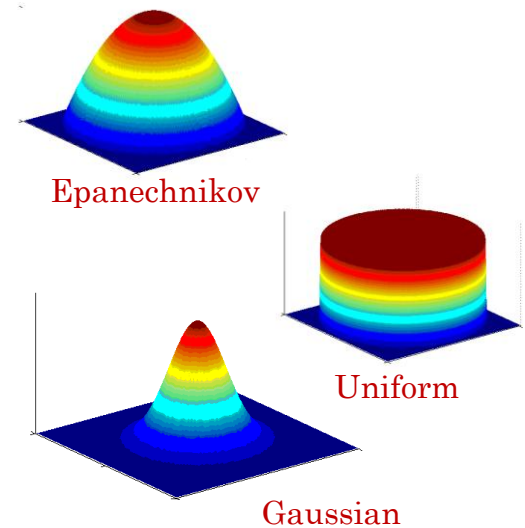
L'algoritmo Mean Shift ricerca i **massimi locali di densità** nello spazio delle feature

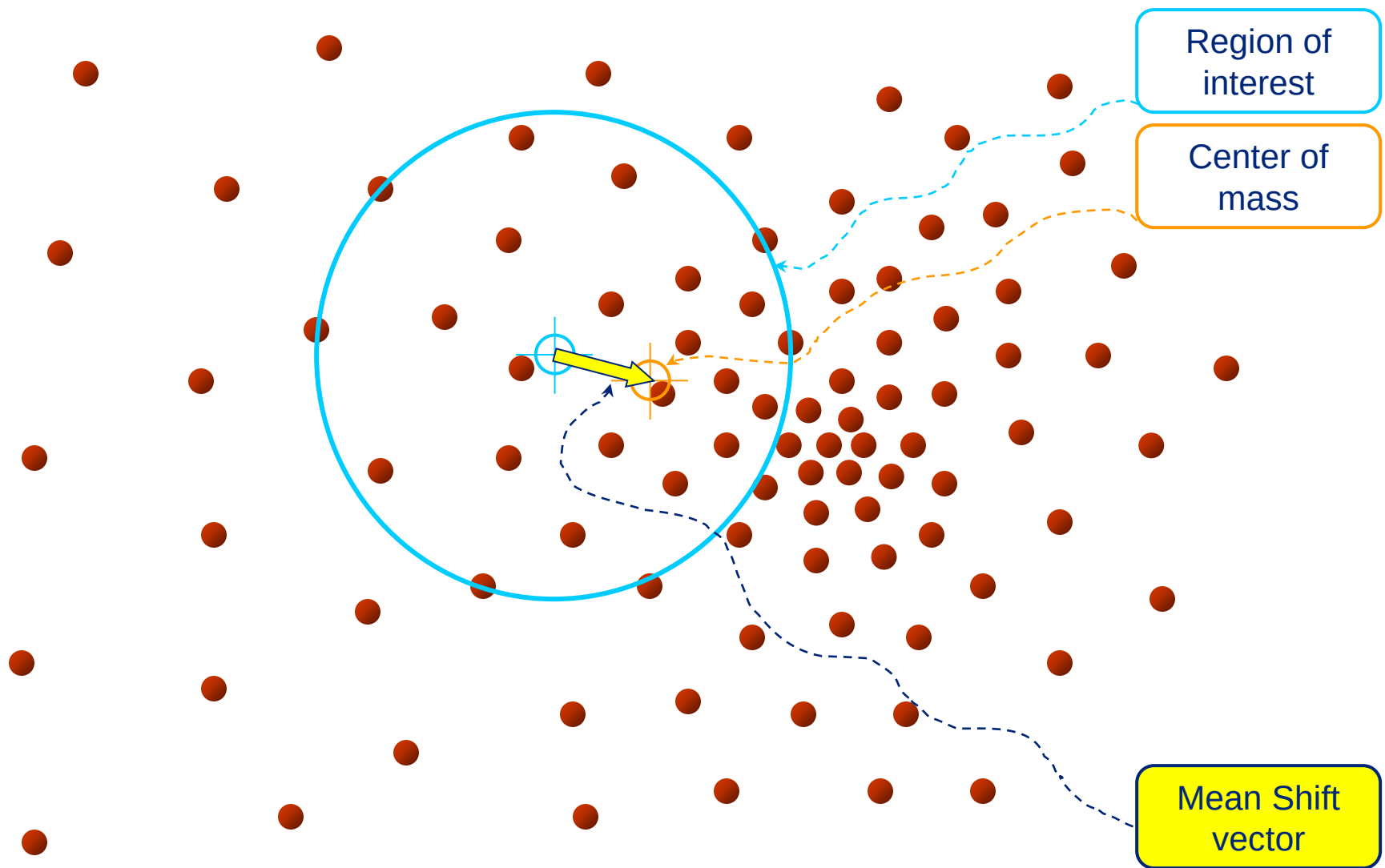


L'algoritmo Mean Shift

- Mean Shift è un algoritmo iterativo che considera lo spazio delle feature come una densità di probabilità empirica.
- L'input all'algoritmo è un insieme di punti che possono essere considerati come **campioni estratti dalla densità di probabilità** in oggetto.
- Se esistono **regioni «dense»** (cluster) di punti, esse corrispondono alla **«moda»** (massimo locale) della distribuzione di probabilità. Scopo dell'algoritmo è individuare questi punti di **massimo locale**.
- Ad alto livello l'algoritmo può essere descritto come segue:

1. Fissa una **finestra** attorno a ciascun punto;
2. **Calcola la media** dei dati all'interno della finestra;
3. **Sposta** la finestra centrandola sulla nuova media;
4. **Ripeti** i passi precedenti fino a convergenza





Obiettivo: Trovare la regione più densa

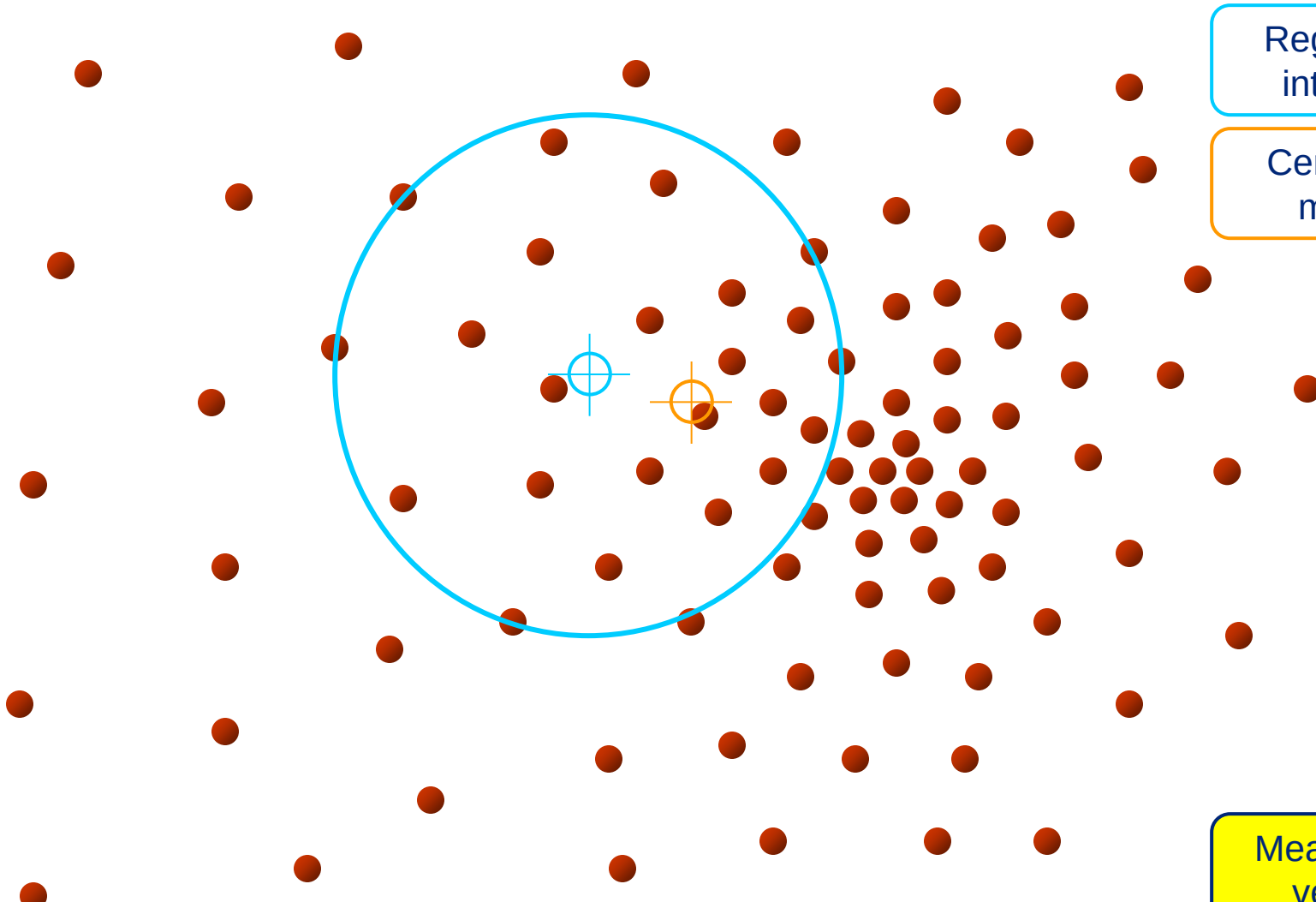
From “Mean Shift Theory and Applications”, presentation for “Advanced Topics in Computer Vision” course, Weizmann Institute.

Region of
interest

Center of
mass

Mean Shift
vector

Obiettivo: Trovare la regione più densa

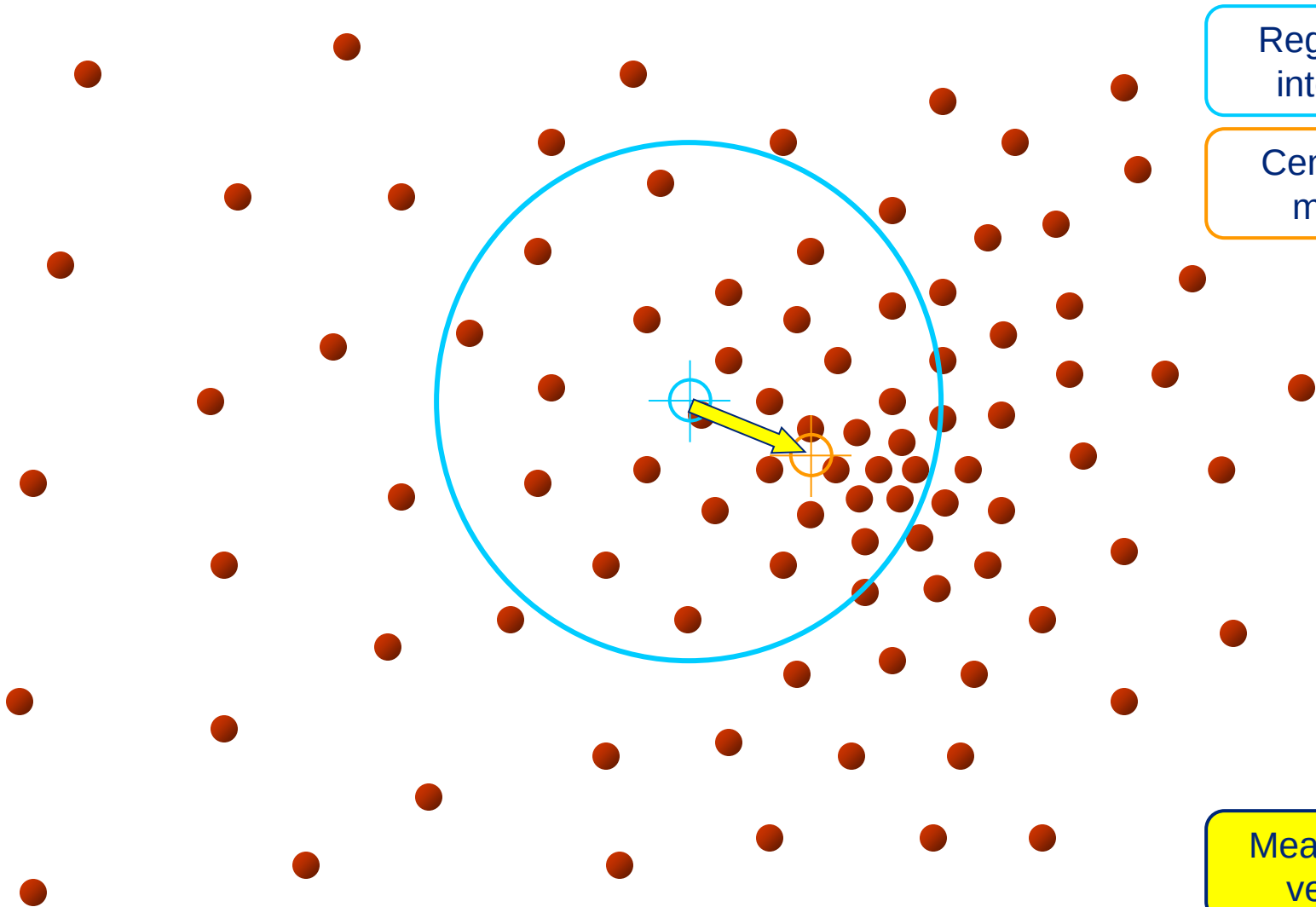


Region of
interest

Center of
mass

Mean Shift
vector

Obiettivo: Trovare la regione più densa

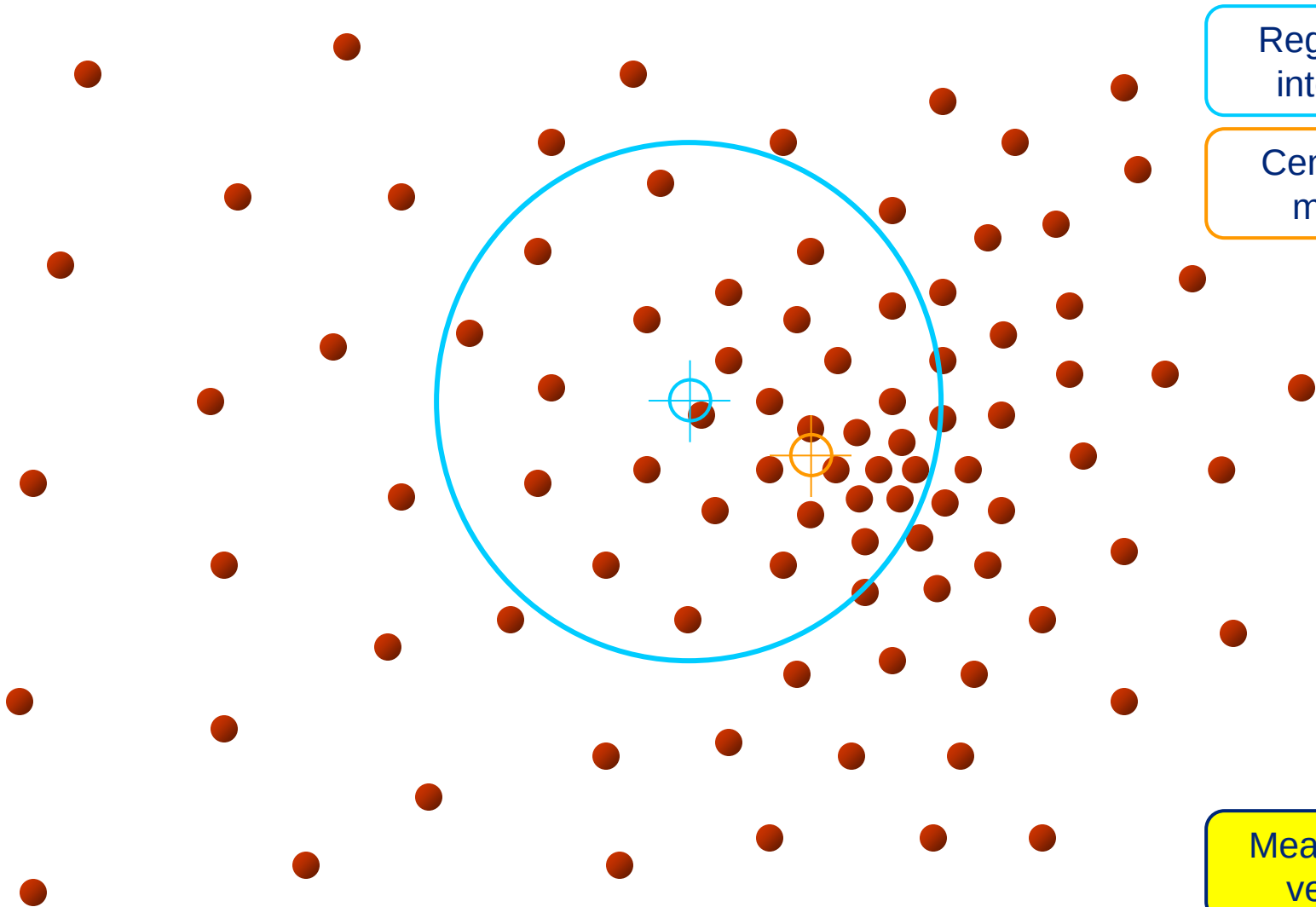


Region of
interest

Center of
mass

Mean Shift
vector

Obiettivo: Trovare la regione più densa

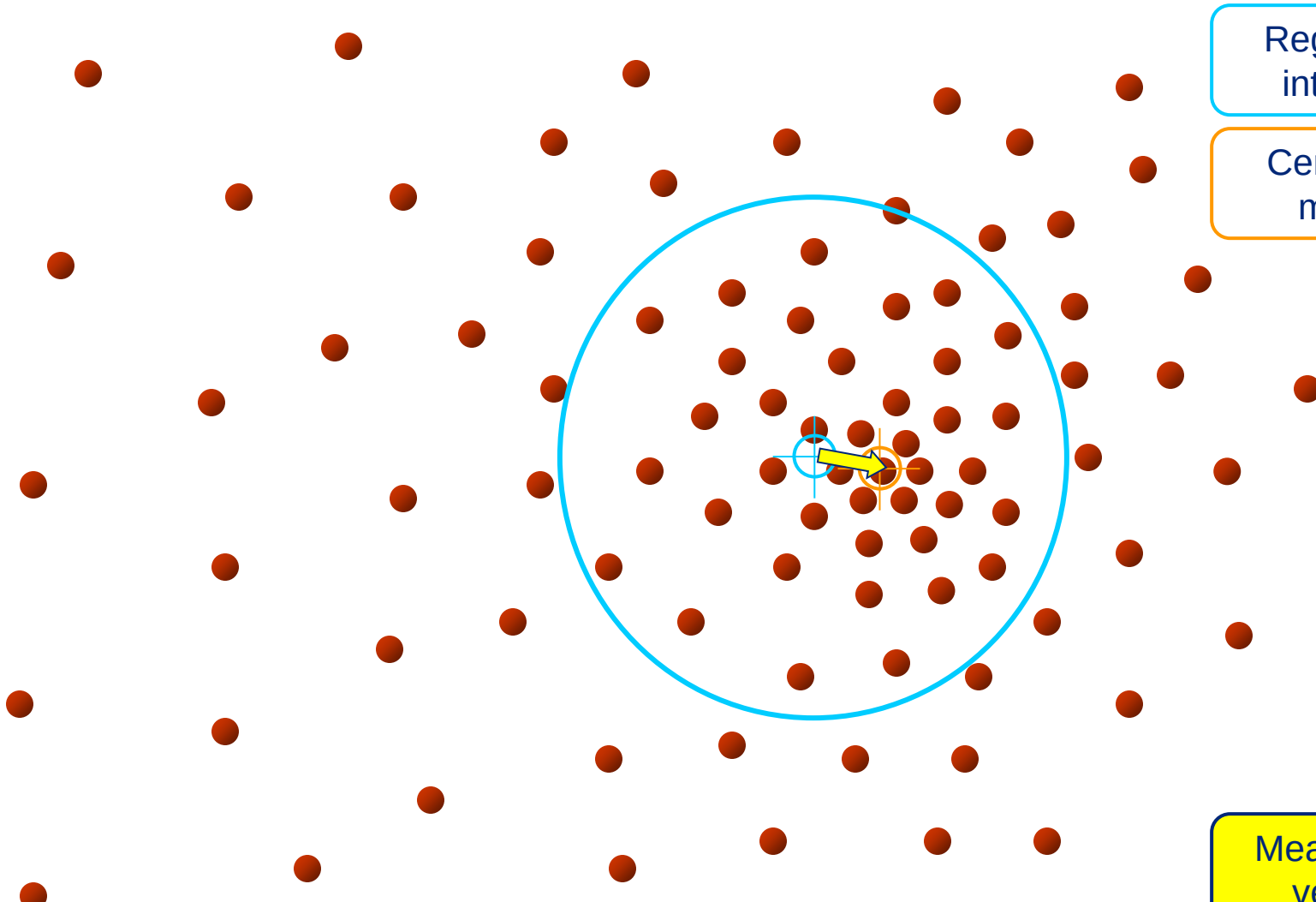


Region of
interest

Center of
mass

Mean Shift
vector

Obiettivo: Trovare la regione più densa

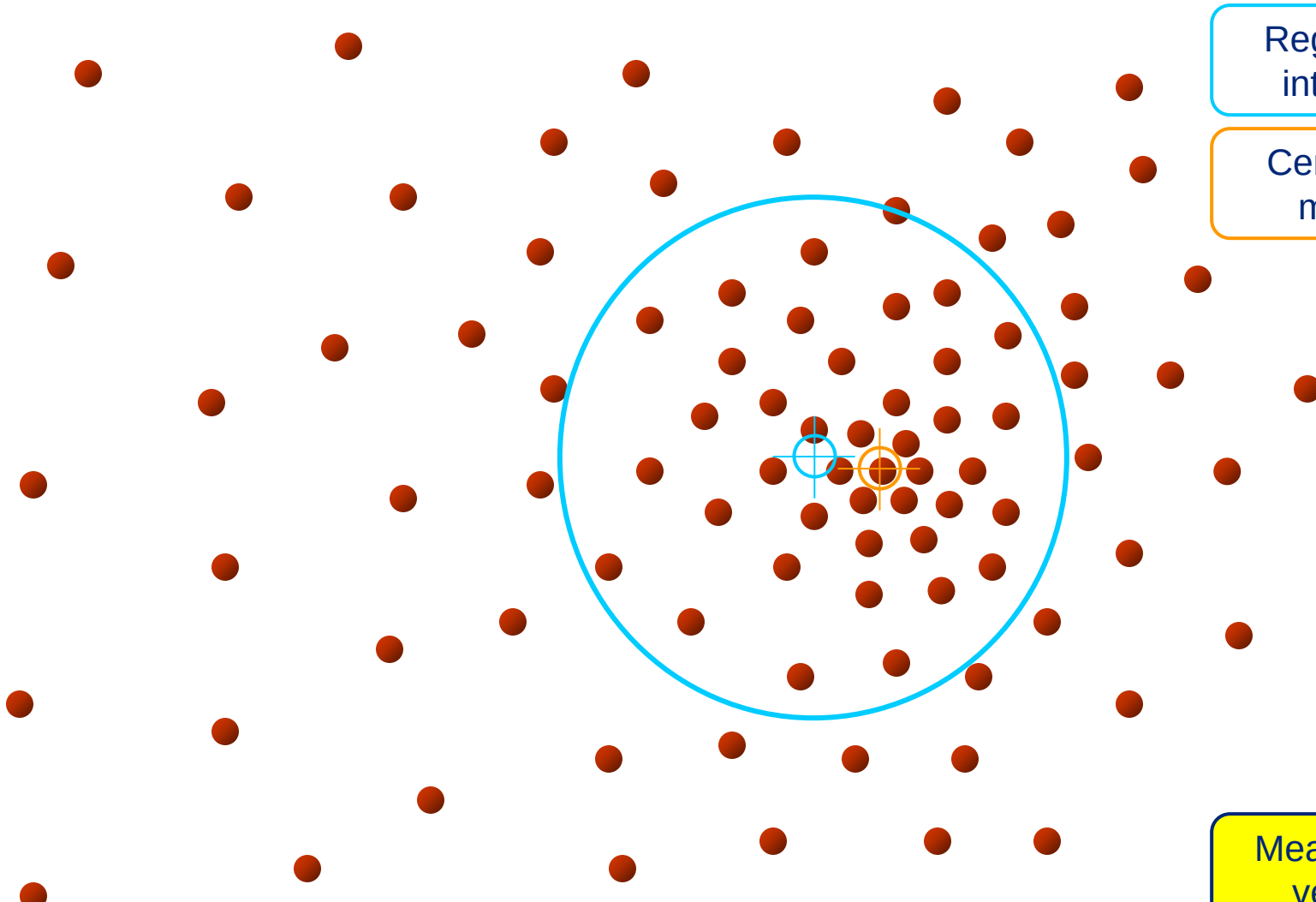


Region of
interest

Center of
mass

Mean Shift
vector

Obiettivo: Trovare la regione più densa



Region of
interest

Center of
mass

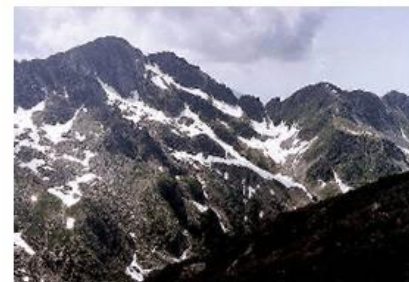
Obiettivo: Trovare la regione più densa

Segmentazione con Mean Shift

- Cluster: insieme dei punti che si trovano nello stesso “bacino di attrazione” di un massimo locale.

Algoritmo di segmentazione:

1. **Codificare** opportunamente l'immagine tramite feature;
2. **Inizializzare** le finestre di osservazione (es. gaussiane) in corrispondenza di ciascun feature vector;
3. Eseguire l'algoritmo **mean shift** fino a convergenza;
4. **Fondere** le finestre (pixel) che si riferiscono allo stesso massimo locale.



Mean Shift: risultati



Mean shift: vantaggi e svantaggi

- Vantaggi:
 - Ricerca **automatica** dei massimi locali
 - È necessario impostare **un singolo parametro** (dimensione della finestra di osservazione)
 - È in grado di rilevare cluster di **forma arbitraria**
 - Tecnica **generica**
- Svantaggi
 - Determinare la **dimensione** della finestra ottimale non è semplice
 - Non adatto a vettori di feature a **dimensionalità elevata**.